Doc4 — Technical Design & Developer Guide: HabbitQuest

Nov 3, 2026 · @Rahul

1. Getting started

You can run the whole app locally in about 15 minutes: PostgreSQL and the API in `backend/`, the React app in `frontend/`.

Prerequisites: Python 3.12 with uv, PostgreSQL 17 (its `bin` folder on PATH), Node.js 22 and git. A Groq key is optional: without one, the AI features use a built-in stand-in.

```
git clone https://github.com/rahulwarekar20/habbit-quest.git
cd habbit-quest/backend
uv sync --locked
uv run python scripts/bootstrap_local.py      # local PostgreSQL, databases
and a git-ignored .env
uv run python -m uvicorn habitquest_backend.main:app --reload --host
127.0.0.1

cd ../frontend
npm install
cp .env.example .env.local                     #
VITE_API_URL=http://127.0.0.1:8000, VITE_DEV_LOGIN=true
npm run dev                                    # http://localhost:5180
```

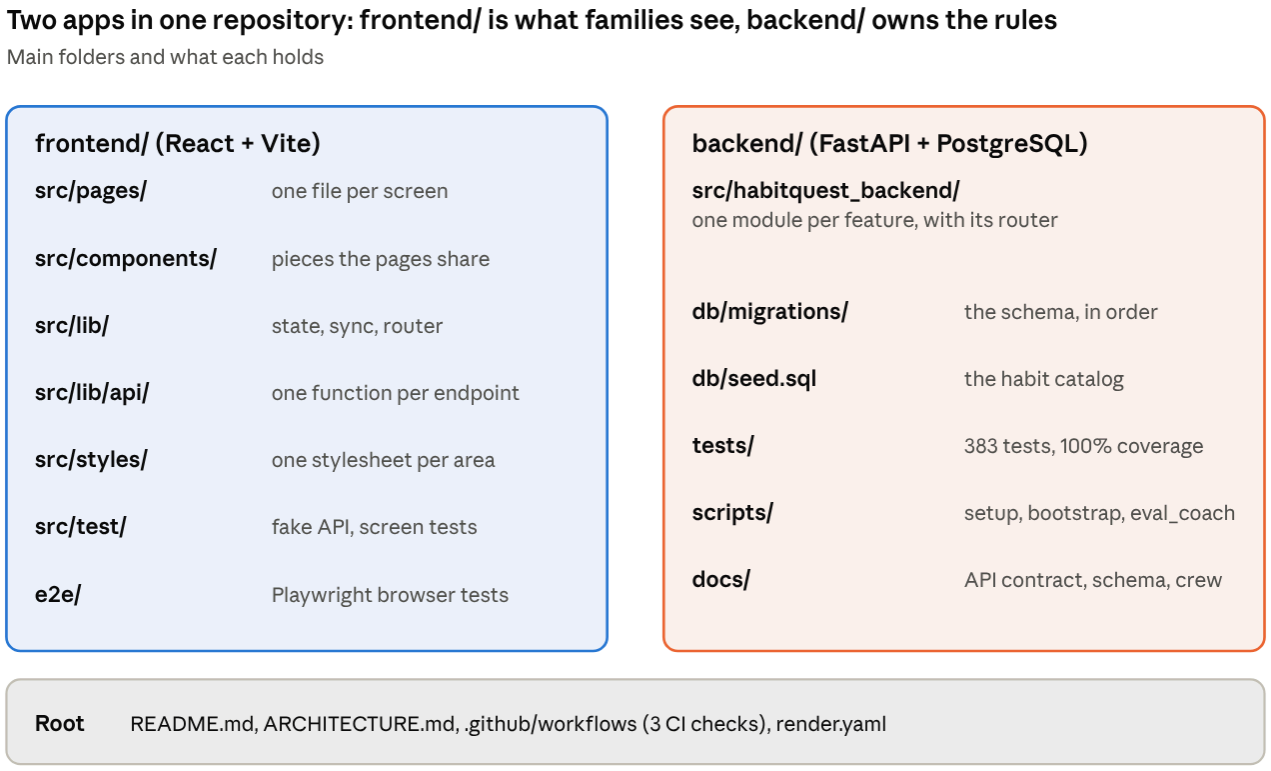
Open `http://localhost:5180` (localhost, not 127.0.0.1) and use **Developer login**: username `parent`, or `parent:<label>` for a separate test parent, with the `DEV_LOGIN_PASSWORD` from `backend/.env`.

Task	Command	Run in
Run the API	`uv run python -m uvicorn habitquest_backend.main:app --reload`	backend/
Backend tests at 100% coverage	`uv run python -m pytest` (on Windows set `PYTHONUTF8=1`)	backend/
Lint, format and types	`uv run ruff check src tests scripts`, `uv run ruff format --check src tests scripts`, `uv run python -m mypy src`	backend/

Task	Command	Run in
Score the coach with real models	`uv run python scripts/eval_coach.py`	backend/
Run the app	`npm run dev`	frontend/ or the root
Logic and screen tests	`npm test`	frontend/ or the root
Browser tests	`npm run e2e` (first `npx playwright install chromium`)	frontend/ or the root
Type-check and build	`npm run build`	frontend/ or the root

2. Repository map

One repository, two apps: `frontend/` is what families see and `backend/` is the API, database and AI. `ARCHITECTURE.md` at the root shows how they fit together.



repository map · frontend/, backend/ and the root

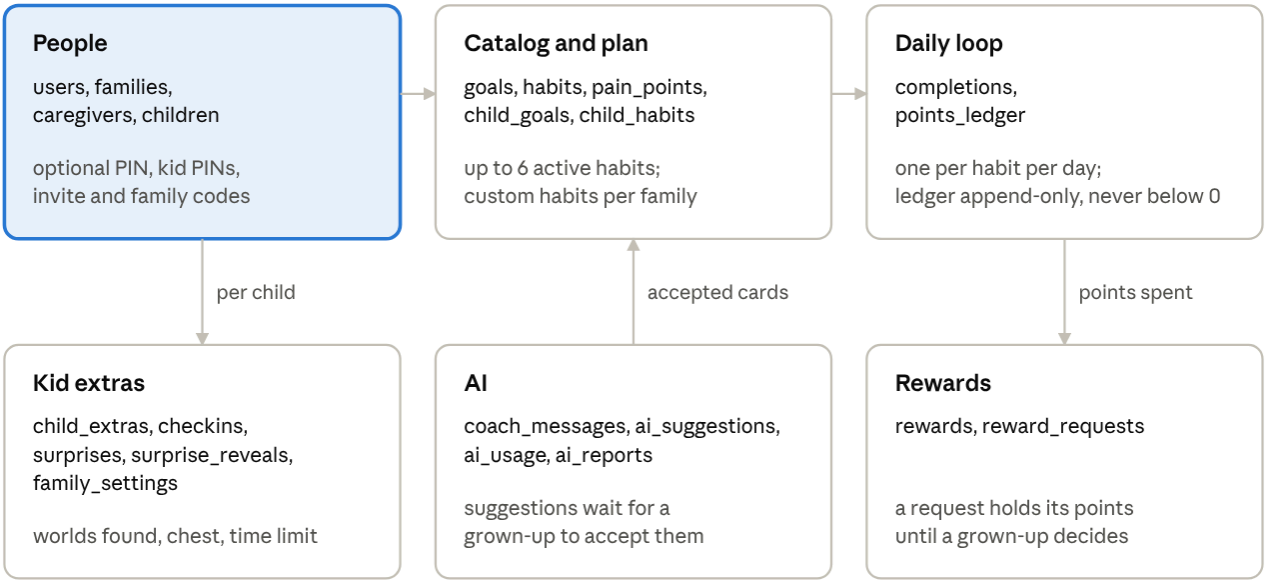
Every file opens with a comment saying what it is for, and every exported function, component and class has one too. Settings live in `backend/.env` (from `.env.example`) and `frontend/.env.local`; neither is committed.

3. Data model

All family data lives in PostgreSQL, schema `hq`: 22 tables created by the SQL files in `backend/db/migrations/`, applied in order by `scripts/setup_db.py` (and on every Render start). `backend/docs/schema.md` has diagrams of every table.

Everything hangs off a family and its children, fenced by row-level security

The 22 tables of schema hq, by area



Every family table has a row-level security policy; kid tokens use a narrower role that sees one child.

data model · 22 tables in 6 areas

Rules the database enforces

- Row-level security on every family table: a caregiver sees only their families; a kid token sees only that child.
- The points ledger is append-only, and a trigger refuses any entry that would take a balance below zero.
- One completion per habit per day; a kid can only submit today's, and only as `submitted`.
- The catalog is read-only to the API; a family's custom habits (ids starting `my-`) are visible only to that family.

Rules in Python, one module per feature

Module	Rules
`onboarding.py`	Children, plans of up to 6 habits, age and tier checks, routine times
`daily.py`	Today's quests, approvals, boosts (golden, streak, double, welcome back, up to 3x)
`journey.py`	Sparks (a quest's tier, at most 6 a day), the six worlds, landmarks, quest stars
`rewards.py`	The reward shop, requests that hold points, mystery boxes, redemptions
`extras.py`, `checkins.py`	Chest, wardrobe, time limit, family settings, check-ins, surprises
`habits.py`	The whole catalog and custom habits
`accounts.py`	Sign-in, families, caregivers, invite and family codes, PINs, export and deletion

Random outcomes (golden quest, chest prizes, mystery boxes) are seeded per child and day or drawn on the server, so the browser cannot choose them.

4. Frontend state and the API client

`StoreProvider` in `frontend/src/lib/store.tsx` holds who is signed in, the family and children from the API, kid mode and the PIN prompt; pages read it with `useStore()` and call the API through `lib/api`.

```
const { askPin, reload } = useStore();
if (!(await askPin(`Approve "${item.title}" for ${item.child_name}.`)))
return; // at once if the family has no PIN
await approveCompletion(item.completion_id);
// lib/api: one function per endpoint
await reload();
```

- `lib/api/` has one file per area; `client.ts` adds the caregiver or kid token, turns failures into an `ApiError` whose `detail` is written for people, and waits for a sleeping server.
- The caregiver token lives in `localStorage` (8 hours); the kid token in `sessionStorage`, so kid mode is per tab.

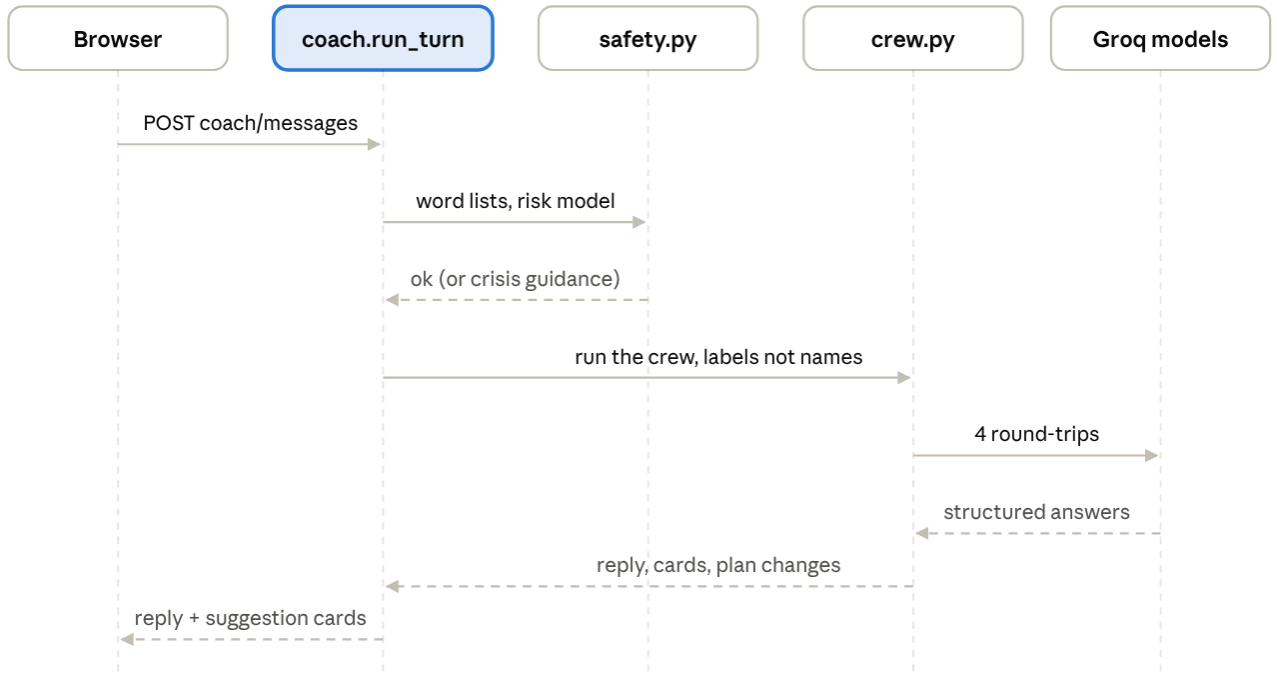
- `lib/sync.ts` copies the API's data into the model the pages draw from; the browser keeps only which celebrations it has shown.
- A 401 ends the session and sends the user to sign-in.
- Routes are hash routes (`#/parent`), read with `useRoute()` from `lib/router.ts`.

5. The API and the AI crew

`main.py` builds the FastAPI app from one router per feature. Each endpoint gets the caller from the bearer token (`auth.signed_in`) and runs its SQL inside `db.transaction`, which sets the database role and the caller's id so row-level security applies.

A coach turn: risk checks first, then four round-trips to the model

One parent message; solid arrows are calls, dashed arrows are answers



coach request sequence · 5 participants

Agents on the same row run at the same time, so a crew turn costs four model round-trips in a row. `backend/docs/coach-crew.md` has the full workflow and what happens when each step fails.

Building blocks

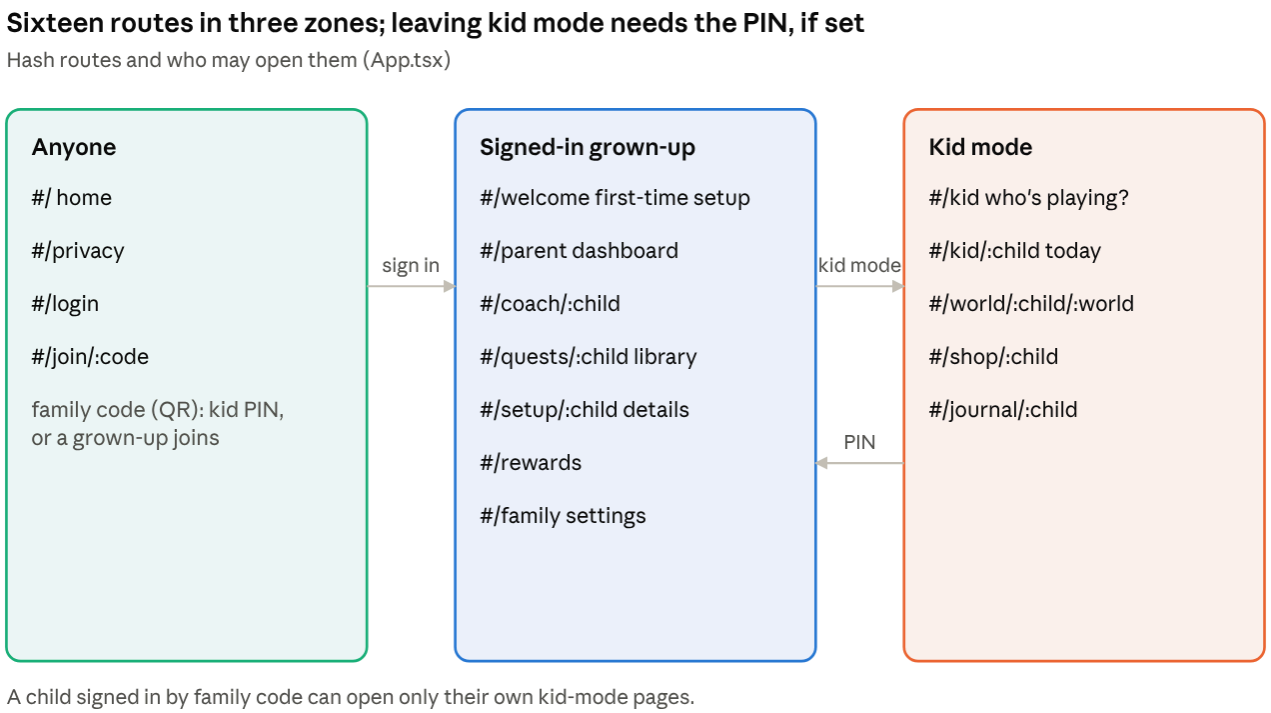
Module	What it does
`crew.py`	The coach crew: triage, research, progress, planner, fit check, safety review (LangGraph)

Module	What it does
`coach.py`, `coach_tools.py`	One coach turn (risk checks, labels for names, deadline, fallback) and the single-agent engine
`guidance.py`	Splits `guidance.json` into 56 passages and ranks them with BM25
`safety.py`	Word lists and the safety model, for kid text and parent messages
`insights.py`	Quest planner, weekly review, buddy lines
`llm.py`	The OpenAI-compatible chat model (LangChain) and one shared HTTP client
`limits.py`	Sign-in, PIN and daily AI limits
`tracing.py`	Optional, redacted coach traces to LangSmith

Tests never call a model: they replace `llm.http_client`, or answer each crew agent by its schema's name. `scripts/eval_coach.py` scores both engines with real models.

6. Frontend structure

The app is one page with hash routes (`lib/router.ts`), so it runs on any static host. `App.tsx` decides which page each route shows and who may open it.



A child signed in by family code can open only their own kid-mode pages.

route map · 3 zones, 16 routes

Conventions

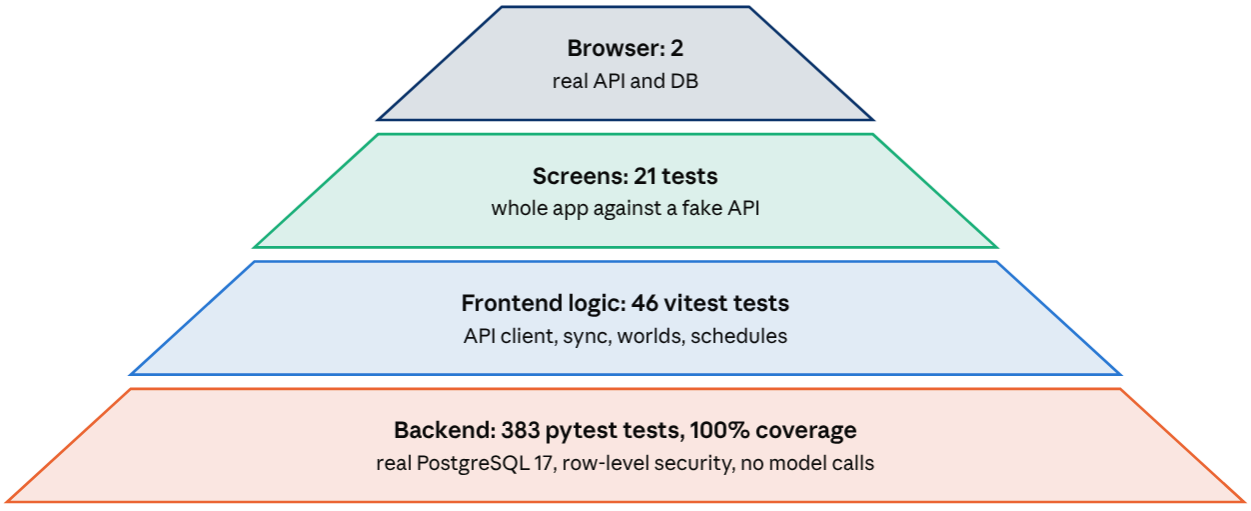
- One file per page in `pages/`; shared pieces in `components/`.
- Styles: one stylesheet per area in `styles/`, loaded in order by `index.css`; colours and sizes come from the tokens in `01-tokens.css`.
- Every API call goes through `lib/api`; pages never call `fetch`.
- Error messages come from the API's `detail`, which is written for people.
- Keep the dev server on port 5180: the API's CORS setting and the Google client allow only that origin.

7. Testing

The backend has one large suite at 100% coverage; the frontend has three layers, from fast logic tests to browser tests against the real API. CI runs them all on every change.

Most tests are fast; two browser journeys prove the whole stack

Automated tests by layer; CI runs every layer on every change



Not in CI: scripts/eval_coach.py scores the coach against realistic parent messages with real models.

test pyramid · 383 backend, 67 frontend, 2 browser tests

Conventions

- Backend tests need a database whose name ends in `_test`; fixtures delete its data. Coverage must stay at 100% (statements and branches).
- Backend tests never call a model; they fake the HTTP client or each agent's answer.
- Screen tests (`*.screen.test.tsx`) render the whole app against `src/test/fakeServer.ts`; a request the fake does not know fails the test.

- Browser tests (`frontend/e2e/`) sign in as a new development-login parent each time, so they never share a family.
- Find elements by role and name, as a person would.

8. How-to recipes

Most changes touch one backend module and its test file, and one function in `frontend/src/lib/api` .

Add a catalog habit

1. Add a row to `backend/db/seed.sql` (id, goal, wording, icon, part of the day, ages, tier, points); the seed is upserted on every start.
2. Optionally draw it in `frontend/src/components/HabitArt.tsx` ; otherwise its emoji is shown.
3. Run the backend tests.

Change the default reward shop

1. Add a migration that redefines `hq.default_rewards` ; it fills the shop of each new family.
2. Existing families add rewards themselves in Rewards & limits.

Add a guidance passage

1. Add a `##` section, with the ages it applies to, to a document in `backend/src/habitquest_backend/guidance.json` .
2. Add a test in `tests/test_guidance.py` that a typical question finds it.

Change a crew agent

1. Edit its prompt and schema in `crew.py` ; its model and effort are in `ROLES` , or in `CREW_ROLES` without a code change.
2. Update `tests/test_crew.py` , then score the change with `scripts/eval_coach.py` .

Add an endpoint

1. Add it to its feature's router with a docstring (it becomes the API docs), a `StrictModel` request body and its SQL inside `db.transaction` .
2. A new table needs a migration with row-level security.
3. Add backend tests, one function in `frontend/src/lib/api/` , and a line in `backend/docs/frontend-integration.md` .

9. Coding standards and troubleshooting

Standards

- Python: ruff for lint and format, mypy, type hints throughout, 100% test coverage.
- TypeScript: strict mode; `npm run build` type-checks everything, the tests included.
- A comment at the top of every file and on every export, saying what it is for.
- Never commit `.env` files or secrets; the frontend gets no API keys.
- One pull request per change, merged only when CI is green.

Troubleshooting

Problem	Fix
`pg_ctl` cannot start the server	Another program may hold the port: start the cluster on a free port and update `DATABASE_URL` in `backend/.env`
Sign-in fails with a CORS error	Open the app at `http://localhost:5180`, not 127.0.0.1
Backend tests fail on Windows with encoding errors	Set `PYTHONUTF8=1`
"Waking up the server" for up to a minute	The free hosted API was asleep; it answers once it has started
The coach answers with keyword matches only	`AI_MODE` is `fake`, or the model call failed: check `AI_MODE` and the `OPENAI_*` settings
The model provider returns 429	Groq's free tier allows about two coach turns a minute: wait, or use a paid tier

FAQ

- **Why can't the coach change a plan itself?** Every AI change waits for a grown-up to accept it; no agent has a tool that writes.
- **Where are the endpoints documented?** At `/docs` on the running API, and in `backend/docs/frontend-integration.md`.
- **How do I test as a second family?** Sign in with the developer login as `parent:<any label>`.